Semantic-Guided Adaptive Dataframe Execution across TypeScript and WebAssembly

Touhidul Alam Seyam[1*]

[1*]Department of Computer Science & Engineering, BGC Trust University Bangladesh, Bangladesh
ORCID: 0009-0007-7512-1893.

Corresponding author(s). E-mail(s): touhidulalam@bgctub.ac.bd;

Abstract

A dataframe system that crosses the JavaScript to WebAssembly boundary must recover the cost of validation, packing, calls, and reconstruction. This paper presents a semantic-guided execution design in `bun_panda`, an eager TypeScript dataframe library with a 7,359-byte Rust/WebAssembly kernel module. A differential oracle executes 2,500 deterministic valid and boundary cases against pandas 3.0.5. The adaptive implementation agrees in 2,500 cases (100.00%), and forced TypeScript, forced WebAssembly, and adaptive execution agree in all 2,500 cases. A fresh-process ablation contains 720 processes and 14,400 timed calls. Full numeric sorting and fused grouped aggregation show statistically supported WebAssembly gains at every tested scale, while mask compaction loses and large top-k selection is unresolved. A separate matched study contributes 400 fresh processes and 4,000 timed calls across `bun_panda`, Arquero, Danfo.js, nodejs-polars, and DuckDB-Wasm, with operation-only and load-plus-operation scopes. A 200-process UCI Bank Marketing validation changes the synthetic ranking and confirms a substantial join deficit. Finally, 180 isolated Chromium, Firefox, and WebKit contexts execute 10,800 kernel calls. WebAssembly sorting is 1.78× in Chromium, 1.28× in WebKit, and 0.47× in Firefox. The results support a backend policy keyed by operation, shape, representation reuse, and runtime, rather than by language alone.

Keywords: dataframe, WebAssembly, TypeScript, Bun, pandas compatibility, performance evaluation, reproducible software

1 Introduction

Dataframe APIs have accumulated hundreds of methods for indexing, mixed data types, missing values, reshaping, joins, grouped reductions, and I/O. Their observable behavior also includes labels, ordering, dtypes, exceptions, and copy rules. Petersohn et al. identify this mix of flexible semantics, eager execution, and API redundancy as a scalability problem [20]. Modin shows that a pandas-compatible layer can be decomposed over a smaller execution core, provided that the system preserves metadata and semantics [21].

JavaScript and TypeScript fit applications that already use web technologies, but boxed objects and dynamic values add work to numeric loops. Wasm supplies validated low-level code and linear memory [7, 24]. It does not remove the cost of recognizing data, copying it into linear memory, calling the kernel, and rebuilding JavaScript results. Wasm can remain slower than native code [9], while DuckDB-Wasm shows that it can also support a substantial analytical engine [12]. The relevant question for a dataframe library is which operations recover the boundary cost.

We study that question through `bun_panda`, an MIT-licensed dataframe library for Bun [18]. It exposes pandas-inspired DataFrame and Series objects and delegates a small set of numeric operations to a 7.2 KiB Rust/Wasm module. It is row oriented rather than a columnar query engine, and it does not claim complete pandas compatibility. The implementation keeps irregular dataframe behavior in TypeScript and uses Wasm only for call shapes that can be represented as compact numeric buffers.

The paper makes five contributions:

1. a layered mixed-backend architecture that keeps labels, null behavior, ordering, and errors in a common semantic layer;
2. a differential oracle that separates API-name coverage from values, labels, order, dtype families, exceptions, and mutation behavior;
3. a fresh-process WebAssembly ablation with randomized execution order, output digests, and hierarchical confidence intervals;
4. a correctness-gated five-system comparison that separates operation cost from input conversion and engine initialization; and
5. a cross-engine browser study and distribution analysis covering worker execution, WebAssembly feature detection, initialization, and compressed artifact size.

The results vary by input, operation, and runtime. Synthetic counting and filtering contain favorable cells, while the public workload changes the ranking and joins expose the cost of row reconstruction. WebAssembly helps full sorting on Bun, Chromium, and WebKit, but the same kernel loses in Firefox. The study therefore treats backend choice as an empirical systems decision rather than a fixed property of WebAssembly.

2 Background and related work

2.1 Dataframe semantics and compatibility

pandas established Series and DataFrame as labeled one- and two-dimensional data structures in the Python ecosystem [13, 19]. Its public API is not a formal behavioral

specification: documentation, implementation, dtype rules, index alignment, warnings, exceptions, and version history jointly determine observable behavior. Consequently, compatibility has several separable dimensions: name presence, callable signature, result values, labels and ordering, dtype/null representation, exception behavior, and mutation or copy semantics.

Recent comparative work evaluates dataframe libraries along workload and usability dimensions rather than assuming that a shared method name implies a shared operation [15]. We adopt that principle. The project's 505-item ledger is a frozen documentation census; test coverage and differential agreement are separate evidence. This distinction is especially important for accessors and advanced I/O functions that may exist to provide discoverable errors rather than full implementations.

2.2 JavaScript analytics systems

Arquero represents tables as column-oriented arrays and offers relational and dplyr-inspired verbs [8]. Danfo.js supplies a pandas-like JavaScript interface, nodejs-polars exposes a native columnar engine, and DuckDB-Wasm embeds a vectorized SQL engine in WebAssembly [10, 12, 22]. These systems differ in representation, initialization, and execution model. We therefore compare a small intersection of matched operations, verify canonical output hashes, and report operation-only separately from load-plus-operation timing.

2.3 WebAssembly boundaries

Wasm provides portable validated code, structured control flow, and bounds-checked linear memory [24]. The host/Wasm interface is intentionally narrow. JavaScript objects and strings do not cross it directly; a library must encode them into bytes or typed arrays. For an operation over n rows, a useful first-order model is

$$T_{\text{wasm}}(n) = T_{\text{dispatch}} + T_{\text{pack}}(n) + T_{\text{kernel}}(n) + T_{\text{unpack}}(n), \tag{1}$$

while the TypeScript path costs $T_{\text{ts}}(n)$. Dispatch is beneficial only when $T_{\text{wasm}}(n) < T_{\text{ts}}(n)$. The inequality depends on data shape, selectivity, output size, runtime optimization, and reuse of resident buffers. It cannot be inferred from the kernel language.

3 System goals and scope

The design has five goals: (G1) a familiar pandas-inspired interface; (G2) deterministic labels and row order; (G3) a TypeScript path for every accelerated operation; (G4) a small Wasm boundary without generated binding glue; and (G5) deployment as a Bun package. Version 0.4.0 does not target pandas binary compatibility or zero-copy columnar storage. Browser support is limited to an asynchronous numeric-kernel entry; the full DataFrame API still requires Bun.

Table 1 records the implemented boundary. "API name" means a callable or intentional error surface, not complete semantic equivalence.

Table 1 Capability boundary in version 0.4.0.

Area	Implemented path	Avail.	Principal limitation
Core frames	DataFrame, Series, GroupBy, reshape, windows	Yes	Eager row materialization
Numeric paths	group IDs, fused aggregate, argsort, mask compaction	Yes	Explicit host-memory copies
CSV/JSON	Bun-native read/write adapters	Yes	No streaming query plan
Excel	`xlsx` adapter	Yes	External package dominates deployment
Parquet	`parquetjs-lite` read/write	Yes	Conversion into library rows
Arrow IPC	Named compatibility entry points	No	No binary IPC engine
Browser kernels	Worker-safe asynchronous ESM loader for sort, filter, and numeric aggregation	Partial	Full DataFrame API remains Bun-only
Rich accessors	Styling, sparse, and struct errors	No	Explicitly outside current scope

Table 2 separates current behavior from possible extensions. In particular, a small Wasm module does not by itself provide browser support, lower memory use, larger-than-memory execution, or GPU acceleration.

4 Architecture

4.1 Layered organization

Figure 1 maps the repository layers. Public objects validate labels and arguments, then call operation-specific modules. Selection and GroupBy modules decide whether an operation can use Wasm. General data remains an array of row objects; the library builds typed storage only for eligible numeric columns. The loader instantiates one Wasm module, copies flat buffers into it, calls an exported C-style function, copies the result back, and resets a bump arena. A loading or shape failure returns `null`, after which the caller uses TypeScript.

4.2 Representation and semantic ownership

Each DataFrame owns normalized rows, an ordered column list, and an index vector. This representation makes heterogeneous cells, label operations, and pandas-like row reconstruction straightforward, but numerical scans encounter object property access and boxed values. A weakly keyed column store builds only requested columns and reuses them while the row snapshot is unchanged. Finite numbers become contiguous `Float64Array` buffers; null and undefined become IEEE-754 NaN sentinels. Typed ingest primes isolated copies of numeric input, while in-place updates invalidate the store.

This cache does not make the library columnar. Every Wasm call still copies typed input into linear memory, and result rows still become JavaScript objects. Arrow

Table 2 Current trade-offs relative to pandas and the Python ecosystem. Claims refer to version 0.4.0.

Dimension	`bun_panda` 0.4.0	pandas and Python ecosystem
Application integration	Uses TypeScript objects directly in Bun applications and can share JavaScript libraries and types.	Python applications use pandas directly; crossing into a JavaScript application requires a process, service, or embedded runtime.
Browser without a server	The `bun_panda/browser` entry loads typed numeric kernels asynchronously from a static Wasm asset or supplied bytes and runs in a dedicated worker. It does not expose the full DataFrame or I/O surface.	pandas can run through a browser Python distribution such as Pyodide, with its Python runtime and package download [23].
Numeric speed	Wasm improves full numeric sorting in the measured range, but slows mask compaction. The ten pandas cases split six wins and four losses.	pandas uses mature native kernels. Performance depends on the operation and representation; this study does not isolate language effects.
RAM use	Row objects coexist with temporary typed arrays and Wasm buffers. Unconstrained process RSS is recorded, but controlled memory ceilings await the Linux run.	Column-oriented storage avoids JavaScript object overhead, although dtype and workload still determine memory use.
Data beyond RAM	No streaming or out-of-core execution plan. The current design materializes rows eagerly.	pandas is mainly in memory, but the Python ecosystem includes partitioned and distributed dataframe systems such as Modin [21].
GPU acceleration	None. WebGPU could become a separate backend, but version 0.4.0 has no GPU kernels or device-memory model.	pandas itself is CPU based; cuDF supplies GPU execution with pandas fallback for supported environments [17].
Ecosystem reach	Direct access to npm packages, browser-facing types, and JavaScript application code is the clearest present advantage.	Python has a much larger scientific-computing and dataframe ecosystem, with broader formats, extensions, and established workflows.

defines a physical format for sharing buffers across languages [1, 5]. `bun_panda` does not yet use that representation and makes no zero-copy claim.

4.3 Flat ABI and memory discipline

The Rust module is built for `wasm32-unknown-unknown` without `wasm-bindgen`. Exports include allocation/reset functions, dense group-ID construction, single- and multi-plan floating-point aggregation, stable argsort, and filter-index compaction. Figure 2 shows the fused aggregation call. Plan-major output avoids one host call per reduction, but values, plan codes, group IDs, output, and counts still occupy separate contiguous regions.

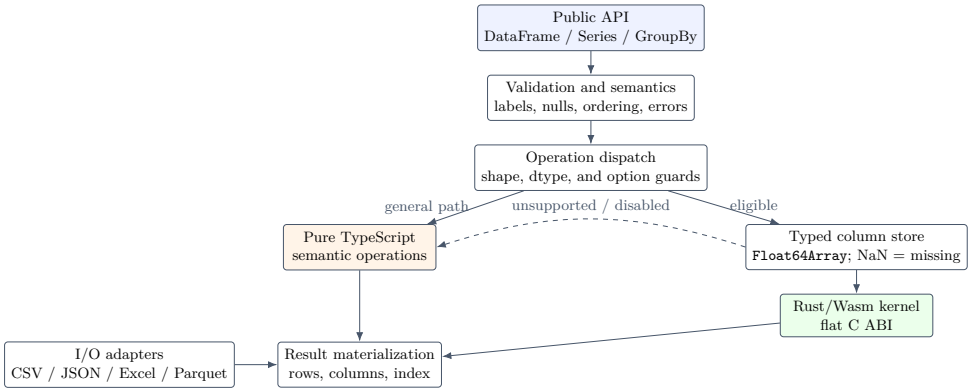


Fig. 1 Concrete architecture. Acceleration is a guarded branch inside an otherwise TypeScript execution path, not a second dataframe engine.

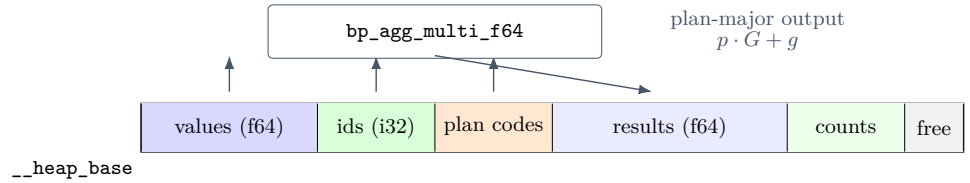


Fig. 2 Wasm linear-memory layout for fused aggregation. The arena begins at the linker-exported heap base; every top-level call copies results before resetting the arena.

During this study, a 250,000-row sequential-kernel test found that the original bump arena began at byte 8 rather than at the linker's exported `__heap_base`. Large allocations could overwrite static module data before memory growth. The corrected allocator starts at the linker symbol. A regression test now runs a 250,000-element argsort followed by mask compaction before performance data is accepted.

4.4 Dispatch logic

Figure 3 summarizes current dispatch. Correctness eligibility is checked before backend preference. The versioned calibration then considers operation, row count, requested top-k, and typed-column reuse. Named GroupBy requires `dropna=true`, supported reductions, and numeric plan columns. Sorting requires one numeric column and default missing-value placement. Filtering remains in TypeScript. `BUN_PANDA_WASM=0` forces TypeScript; `BUN_PANDA_WASM=1` forces only semantically eligible Wasm calls.

5 Compatibility and correctness method

5.1 Name-level census

The audit freezes four public groups from the pandas 3.0.5 documentation: 197 DataFrame methods and properties, 195 Series methods and properties, 75 top-level functions, and 38 GroupBy operations. A source extractor classifies each name as

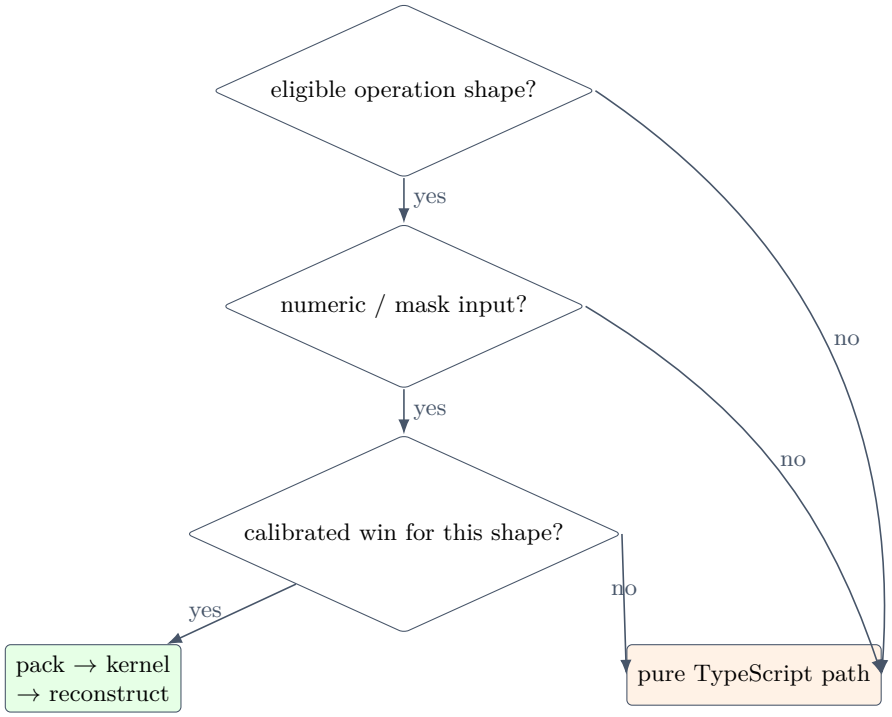


Fig. 3 Correctness-first adaptive dispatch. Eligibility is necessary but does not imply that Wasm is profitable.

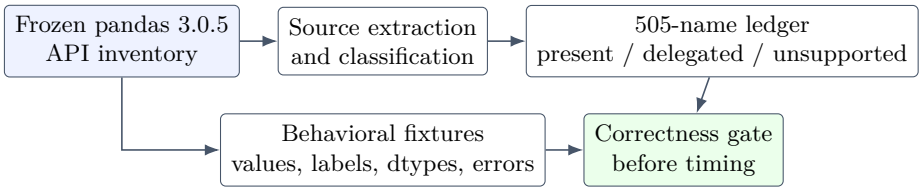


Fig. 4 Compatibility evidence pipeline. Name coverage and behavioral agreement use different denominators and artifacts.

concrete, delegated, or intentionally unsupported and writes a machine-readable Markdown ledger. All 505 names are present in version 0.4.0. The ledger exposes missing entry points. By itself, it says nothing about signatures, dtype fidelity, or edge cases.

5.2 Test oracle and failure handling

The repository test suite uses deterministic fixtures for indexing, alignment, arithmetic, grouping, reshape, windows, I/O, and explicit error cases. The differential corpus adds 200 valid and 50 invalid or boundary cases for each of ten operation families. Both runtimes receive the same rows, column order, index labels, tagged non-finite values, operation, and arguments. The comparator checks status, output kind, values, labels, order, broad dtype families, exception categories, and input preservation. Null,

NaN, NA, and NaT compare as the same missing mask; native dtype spellings remain in the raw artifact. For merges, the comparator requires the same sequence of join-key groups and the same row multiset within each contiguous equal-key group. It does not compare the internal pairing order of duplicate matches because pandas specifies preservation of left-key order, not duplicate-pair order [19].

The corpus also runs `bun_panda` with forced TypeScript, forced eligible Wasm, and adaptive dispatch. A backend mismatch aborts performance analysis for the corresponding input. The current 2,500/2,500 backend agreement was reached after the oracle exposed an eligibility error: forced Wasm had routed `dropna=false` GroupBy through a kernel that always removes missing keys. Other oracle findings led to integer-period validation, missing-label validation in duplicate removal, corrected null ranking, and stable value-count tie order.

The adaptive implementation agrees with pandas in 2,500 of 2,500 observations under these declared comparison rules. This is 100.00% of the tested corpus, not full pandas compatibility. MultiIndex, extension dtypes, warnings, categorical metadata, timezone semantics, and untested argument combinations remain outside the corpus.

6 Experimental method

6.1 Research questions

The evaluation asks: (RQ1) How closely does the tested semantic surface agree with pandas? (RQ2) Which eligible operations benefit from WebAssembly as input size grows? (RQ3) How does the system compare with four JavaScript, native, and WebAssembly alternatives under matched operation and load scopes? (RQ4) Do kernel-level conclusions transfer across browser engines? (RQ5) What distribution cost accompanies the design? (RQ6) Which failures point to changes in representation or dispatch?

6.2 Environment

Experiments ran on an Apple M5 Pro with 18 logical cores, 48 GiB RAM, and macOS 26.5. Versions were Bun 1.4.0, TypeScript 5.9.3, Rust 1.96.0, Arquero 8.0.3, Danfo.js 1.2.0, nodejs-polars 0.26.0, DuckDB-Wasm package 1.33.1-dev57.0 with embedded DuckDB 1.5.4, Python 3.13.13, and pandas 3.0.5. Browser measurements used headless Chromium 151.0.7922.34, Firefox 153.0, and WebKit 26.5 through Playwright. The benchmark process was single-threaded at the library level. Power mode and CPU frequency were not fixed. All host-level performance results in this manuscript are macOS measurements. Reports returned from Windows or Linux reproductions are reviewed as separate validation artifacts and are not pooled into these tables. The exact environment is stored in `paper/data/environment.json`.

6.3 Datasets and matched operations

The primary inputs use a deterministic linear congruential generator. The base schema contains numeric identifiers, values, weights, and revenue; categorical group, city, segment, and bucket strings; a boolean; and bounded-cardinality keys. Variants introduce

skew, additional columns, unique keys, or periodic missing values. A public-data validation path uses the UCI Bank Marketing dataset, DOI 10.24432/C5K306, under CC BY 4.0 [14]. The manifest fixes the source URL, archive and CSV SHA-256 digests, 45,211-row count, field mapping, and seed-independent transformations. Public-data correctness passed for every adapter; its timing is not mixed with the synthetic results.

The legacy Arquero comparison contains 82 cases: seven grouped reductions, 34 filtering, sorting, and top-k pipelines, 32 counting variants, and nine joins. It remains useful as a broad exploratory profile. The principal cross-system study instead fixes four operations available in all five systems: grouped sum, filter then top-100 sort, value counts, and inner join. It runs at 10,000 and 50,000 rows under two scopes. The operation scope starts after each system has constructed its table. The load-plus-operation scope includes construction or registration but excludes process startup and package import. Each result is canonicalized to rows with explicit missing-value tags and compared by SHA-256 digest against a pure TypeScript reference.

6.4 Timing protocol

The legacy Arquero and pandas comparisons use three warm-up executions followed by eight measurements in each of three rounds. Their reported value is the median round mean, so these comparisons remain exploratory. The WebAssembly ablation uses fresh processes. Each workload, scale, backend, and replicate runs in a new Bun process. The controller randomizes all 720 tasks with a recorded seed. Each process constructs typed input, performs five warm-up calls, then records 20 calls. The final output is fully materialized and hashed with SHA-256. Inputs use paired seeds across backends, and all three digests must agree.

The experimental unit for the ablation is the process. For each effect, the analysis resamples processes and then iterations within the selected processes. It reports a 95% hierarchical percentile bootstrap interval from 2,000 resamples [6, 11]. A point estimate counts as a win only if the entire interval is above one. This design retains 14,400 iteration records nested inside 720 processes. Warm-up can still be non-monotonic [4].

The five-system study randomizes 400 fresh process tasks. Each task performs three warm-ups and ten timed calls. Five process replicates cover every system, workload, scope, row count, and seed combination, yielding 4,000 timed calls across 16 matched workload-scope-size cells. Reported values are medians of process means. The limited process count supports descriptive comparison, not confidence claims.

The browser study creates a new browser and dedicated module worker for every engine, scale, and replicate. The page is served with cross-origin isolation headers. Each worker instantiates the same WebAssembly asset, checks feature support, performs three warm-ups, and records ten TypeScript and ten WebAssembly calls for stable argsort, filter-index compaction, and grouped sum. Thirty replicates at 10,000 and 100,000 rows across three engines yield 180 isolated contexts and 10,800 timed calls. Output hashes must match within every context. Browser speedup is the median across 30 context ratios at 100,000 rows. A deterministic 2,000-resample bootstrap over contexts supplies the reported 95% intervals.

For ratios, define speedup as

Table 3 Arquero 8.0.3 comparison at 25,000 rows. Speedup is Arquero time divided by `bun_panda` time.

Family	Cases	Wins	Losses	Geomean speedup
Group-by	7	6	1	1.28×
Sort/filter/top-k	34	34	0	1.93×
Value counts	32	32	0	4.54×
Merge/join	9	0	9	0.34×
Overall	82	72	10	2.15×

$$S = \frac{T_{\text{comparator}}}{T_{\text{bun_panda}}}. \tag{2}$$

Thus $S > 1$ favors `bun_panda`. Aggregate speedups are geometric means of per-case ratios; unlike arithmetic means, this treats reciprocal slowdowns symmetrically.

6.5 Package-size protocol

The consumer tarball is measured with `npm pack --json --ignore-scripts` [16]. A minified Bun-target core bundle externalizes the optional `xlsx` and `parquetjs-lite` packages so that their independent cost is not attributed to core algorithms. A second build targets the browser-only kernel entry and rejects Node or Bun imports in its emitted graph. JavaScript and Wasm are reported raw, under gzip level 9, and under Brotli quality 11.

7 Results

7.1 Behavioral conformance

The 505-name census and the differential result answer different questions. The census records whether a name exists. The oracle finds 2,500/2,500 agreement against pandas 3.0.5 under the declared rules. All ten families agree in their 250 observations. The machine-readable summary records the numeric tolerance, dtype policy, and merge-order contract, while the mismatch ledger is empty for this corpus.

Forced TypeScript and forced eligible Wasm agree in 2,500/2,500 observations, as do TypeScript and adaptive dispatch. This backend equality is a prerequisite for the timing study, not evidence of pandas compatibility.

7.2 Exploratory Arquero profile

Performance varies by workload family in Table 3 and Figure 5. `bun_panda` wins 72 of 82 cases, with an overall geometric-mean speedup of 2.15×. Value-count workloads reach 4.54×, and sort, filter, and top-k workloads reach 1.93×. Group-by reaches 1.28×. `bun_panda` loses all nine merge cases; the family ratio of 0.34× means Arquero is approximately 2.92× faster there.

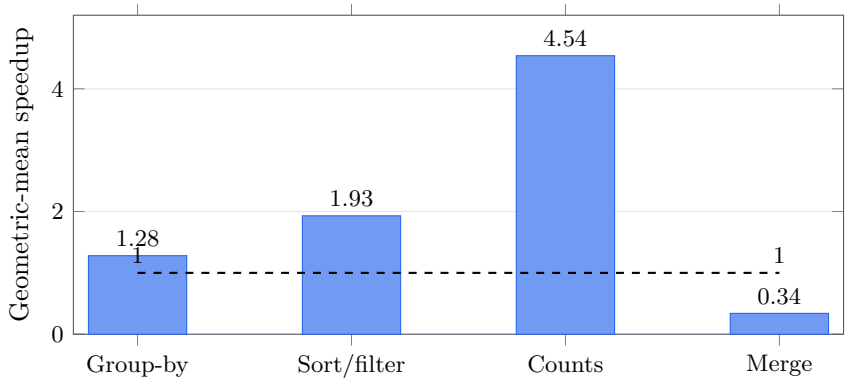


Fig. 5 Family-level performance against Arquero. The dashed line marks parity. Complete per-case values are retained in the JSON artifact.

Table 4 Matched pandas 3.0.5 comparison at 25,000 rows. Ratio below 1 favors `bun_panda`.

Case	bun_panda (ms)	pandas (ms)	bun/pandas
groupby mean	4.67	3.78	1.23
filter + sort top 100	1.42	2.99	0.48
sort top 1000	5.07	2.17	2.33
multi-column sort top 800	10.44	10.43	1.00
value counts: city	0.74	3.20	0.23
value counts: group/city top 10	2.50	5.89	0.43
missing city, retain null	0.91	1.39	0.66
missing city group-by mean	2.71	3.16	0.86
high-cardinality city top 20	20.04	15.18	1.32
high-cardinality user top 100	9.08	33.90	0.27

The implementation choices explain much of this spread. Bounded value counts avoid a general relational plan, and top-k selection can avoid a full multi-column sort. Merge creates and copies row objects, resolves colliding names, and materializes wide results. Arquero's column-oriented join path suits that workload better. Faster joins will require changes to representation and materialization rather than another arithmetic kernel.

7.3 Exploratory pandas profile

Across ten matched cases, `bun_panda` wins 6 and loses four, with a 1.44× geometric-mean speedup. Table 4 gives all cases because the sample is small. The largest advantage is high-cardinality single-key value counts (0.27× time ratio, or 3.73× speedup); the largest disadvantage is single-column top-1000 sorting (2.33× slower). Multi-column top-800 sorting is effectively tied.

This is a cross-runtime comparison between Bun/JavaScriptCore and CPython/-native pandas extensions. It does not isolate language, runtime, data representation,

Table 5 Operation-only median of process means at 50,000 rows, in milliseconds. Lower is better. Each cell contains five fresh processes and 50 timed calls.

Workload	bun_panda	Arquero	Danfo.js	nodejs-polars	DuckDB-Wasm
Grouped sum	2.67	4.22	16.08	1.59	3.06
Filter and top-100 sort	2.05	6.60	114.55	1.56	6.31
Value counts	0.68	2.83	15.97	1.62	3.09
Inner join	24.56	8.16	1807.97	6.09	9.46

Table 6 Load-plus-operation median of process means at 50,000 rows, in milliseconds. Lower is better.

Workload	bun_panda	Arquero	Danfo.js	nodejs-polars	DuckDB-Wasm
Grouped sum	6.96	4.73	19.99	6.72	119.27
Filter and top-100 sort	6.44	6.48	105.85	6.98	123.86
Value counts	4.93	3.84	19.92	6.53	117.98
Inner join	26.39	7.68	2359.89	13.38	126.10

or implementation. pandas remains the semantic reference; these timings answer only whether the selected operation completes faster under the stated environment.

7.4 Matched five-system study (RQ3)

Table 5 reports operation-only medians at 50,000 rows. `bun_panda` is faster than Arquero, Danfo.js, and DuckDB-Wasm on grouped sum, filter plus top-100 sort, and value counts. nodejs-polars is faster on grouped sum and filtering. The ranking reverses for join: `bun_panda` needs 24.56 ms, compared with 6.09 ms for nodejs-polars, 8.16 ms for Arquero, and 9.46 ms for DuckDB-Wasm. Danfo.js is slower on all four cells in this environment.

Load-plus-operation timing in Table 6 changes the interpretation. `bun_panda`, Arquero, and nodejs-polars are close on grouped sum and filtering. DuckDB-Wasm pays about 118 to 126 ms per measured construction and query because the scope includes table registration in a fresh embedded database connection, though not module import or process startup. This is a deliberately literal application boundary, not an estimate of a long-lived DuckDB session. The paired tables show why a single headline speedup would be misleading.

All 16 matched workload-scope-size cells pass the digest gate for all five systems. The study does not claim semantic equivalence outside those four operations. It also avoids averaging incompatible workloads into one score.

The checksum-pinned UCI workload adds 200 fresh processes and 2,000 timed calls across 8 matched cells. Table 7 shows that the synthetic ranking is not universal. At 45,211 public rows, nodejs-polars leads grouped sum, filtering, and value counts, while Arquero leads join narrowly ahead of DuckDB-Wasm. `bun_panda` does not lead

Table 7 Operation-only median of process means on the 45,211-row UCI Bank Marketing workload, in milliseconds. Lower is better.

Workload	bun_panda	Arquero	Danfo.js	nodejs-polars	DuckDB-Wasm
Grouped sum	32.81	29.96	65.54	5.20	19.24
Filter and top-100 sort	5.75	14.23	296.70	2.70	10.71
Value counts	9.14	24.55	79.52	6.54	28.68
Inner join	120.00	27.80	4496.54	66.98	29.30

Table 8 Fresh-process speedups with 95% hierarchical bootstrap intervals. Each cell has 20 processes and 20 measured calls per process.

Workload	Rows	Forced Wasm [95% CI]	Verdict
Fused GroupBy, 4 plans	10,000	1.523 [1.069, 2.306]	win
	100,000	1.442 [1.285, 1.642]	win
	250,000	1.599 [1.323, 1.903]	win
Full numeric sort	10,000	1.713 [1.471, 1.983]	win
	100,000	1.669 [1.298, 2.087]	win
	250,000	1.752 [1.370, 2.186]	win
Top-1,000 sort	10,000	1.458 [1.182, 1.744]	win
	100,000	0.901 [0.737, 1.081]	unresolved
	250,000	0.869 [0.594, 1.258]	unresolved
Boolean-mask filter	10,000	0.199 [0.163, 0.241]	loss
	100,000	0.547 [0.459, 0.651]	loss
	250,000	0.308 [0.269, 0.358]	loss

an operation-only cell, although it remains faster than Arquero on filtering and value counts. Its join time widens to 120.00 ms versus 27.80 ms for Arquero. In load-plus-operation scope, `bun_panda` records 23.46 ms for value counts, lower than all four alternatives, while DuckDB-Wasm again pays per-call registration cost. This public-data reversal is evidence against a library-wide performance claim.

7.5 WebAssembly ablation (RQ2)

Table 8 reports the fresh-process ablation. The effect is mean TypeScript time divided by mean candidate time. Full numeric sorting and fused four-plan GroupBy over reused typed columns win at every scale. Top-1,000 selection wins at 10,000 rows, but both larger intervals cross one. Forced Wasm filtering loses by a wide margin. Adaptive filtering stays in TypeScript, so its intervals center near parity and remain unresolved.

The resulting calibration is conservative. Full numeric sort uses Wasm from 10,000 rows. Fused GroupBy uses Wasm from 10,000 rows only when every plan column is already cached and `dropna=true`. Top-1,000 uses Wasm only at the measured 10,000-row boundary; larger inputs use bounded TypeScript selection. Filtering stays in TypeScript. The environment override can force eligible Wasm calls for experiments, but it cannot bypass semantic eligibility.

Table 9 Median TypeScript-time divided by WebAssembly-time ratio at 100,000 rows across 30 isolated contexts per engine, with 95% bootstrap intervals. Values above 1 favor WebAssembly.

Engine	Stable argsort	Filter indices	Grouped sum
Chromium	1.78 [1.62, 2.09]	1.63 [1.42, 2.10]	2.63 [2.20, 3.05]
Firefox	0.47 [0.42, 0.49]	0.59 [0.50, 0.68]	0.08 [0.07, 0.09]
WebKit	1.28 [1.18, 1.52]	1.19 [0.95, 1.31]	0.38 [0.30, 0.51]

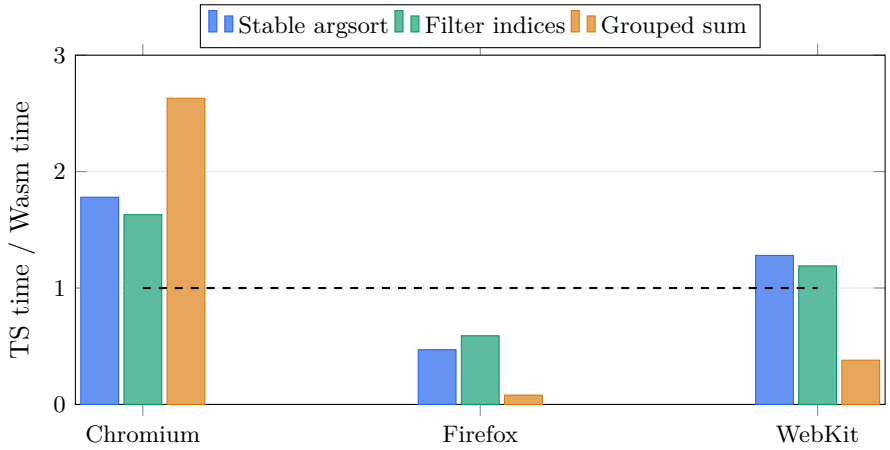


Fig. 6 Browser kernel ratios at 100,000 rows. The dashed parity line exposes the engine-dependent reversal.

7.6 Cross-engine browser execution (RQ4)

All three engines provide WebAssembly, streaming instantiation, threads, SIMD, shared memory, and cross-origin isolation in the tested build. WebKit reports no relaxed SIMD support. Median initialization is 4.75 ms in Chromium, 46.21 ms in Firefox, and 9.14 ms in WebKit. This one-time cost is excluded from the warm kernel ratios in Table 9.

The browser result is not a full DataFrame benchmark. It isolates typed numeric kernels in a dedicated worker and therefore answers whether an already-eligible kernel transfers across engines. It does not include row-object conversion or network download. Even at that favorable boundary, Firefox reverses every conclusion, and WebKit reverses grouped sum. Runtime identity must therefore be part of any browser dispatch calibration.

7.7 Package and bundle size (RQ5)

The npm tarball is 130.7 KiB and expands to 546.1 KiB across 77 entries. The minified full core bundle, with optional Excel and Parquet dependencies externalized, is 180.2 KiB raw and 51.6 KiB under gzip. The browser kernel entry is 2,683 bytes raw,

996 bytes under gzip, and 877 bytes under Brotli. The separate 7,359-byte module compresses to 2,984 and 2,614 bytes, respectively.

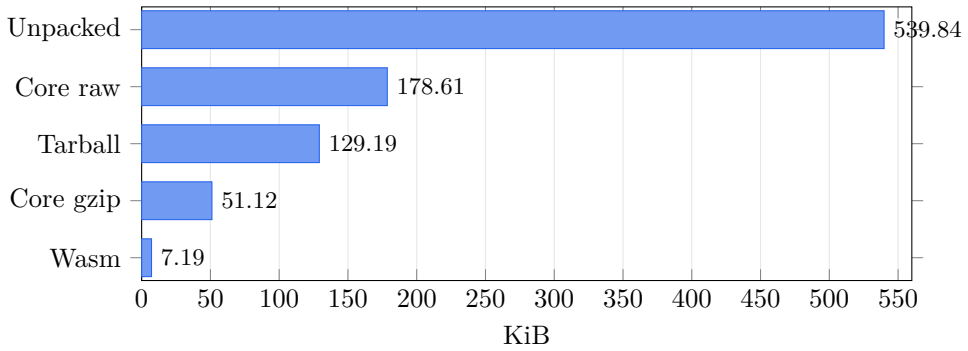


Fig. 7 Distinct distribution artifacts. The scales are not interchangeable: the tarball includes source files, while the core bundle externalizes optional I/O packages.

The full Bun loader reads and instantiates WebAssembly synchronously on the first eligible call. The browser entry has a different contract: it uses `instantiateStreaming(fetch(url))`, falls back to asynchronous byte instantiation when the server sends the wrong MIME type, and accepts preloaded bytes for workers or offline bundles. Static graph checks, byte-level kernel tests, and the three-engine worker study all pass.

8 Discussion

8.1 Architectural findings

Conversion cost matters more than the kernel language in several cases. The same 7.2 KiB module produces a 1.71× full-sort point speedup and a fivefold slowdown for a 10,000-row filter in Bun. Mask compaction is a linear JavaScript loop, and its output still requires JavaScript row gathering. A small Rust loop does not recover the cost of two copies and a host call.

Runtime implementation also matters. At 100,000 rows, stable argsort favors WebAssembly in Chromium and WebKit but loses in Firefox. Grouped sum ranges from 2.56× in Chromium to 0.08× in Firefox. These tests use the same bytes, inputs, worker boundary, warm-up rule, and digest gate. A calibration learned from Bun or Chromium is therefore not portable evidence for another engine.

Algorithm choice explains the 250,000-row top-1000 reversal. The Wasm path performs a full stable argsort. A bounded heap or selection algorithm would change the work from $O(n \log n)$ toward $O(n \log k)$, but moving that algorithm to Wasm would not guarantee a faster result. Dispatch must consider the requested operation, not only dtype.

Row-oriented storage has a visible join cost. A merge produces new heterogeneous row objects and resolves colliding column names. Numeric Wasm kernels do not reduce that materialization work. Columnar join indices could postpone row reconstruction,

but they would also enlarge the semantic core and require additional label and null tests.

The 505-name census creates a large testing obligation. The generated oracle covers ten families and found implementation defects even though only one of 2,500 observations now differs from pandas. The result applies to this corpus, not to untested extension dtypes, time zones, categorical metadata, warnings, or copy-on-write behavior. Adding names is cheap compared with specifying and testing those semantics.

8.2 Revised dispatch policy

Table 10 states the default supported by each measured result.

Table 10 Dispatch changes supported by the measured ablation.

Operation	Recommendation	Evidence
Full numeric sort	Use Wasm from 10,000 rows	Every measured interval is above one
Top-k numeric sort	Use Wasm only at the measured 10,000-row cell	Larger intervals cross one
Boolean mask	Keep TypeScript default	Every forced-Wasm interval is below one
Fused numeric group-by	Use Wasm when typed plan columns are cached	Every reused-column interval is above one
Merge/join	Optimize representation, not arithmetic kernel	0/9 wins vs Arquero
Browser kernels	Calibrate independently for each engine and version	Ratios reverse across Chromium, Firefox, and WebKit

8.3 Interoperability roadmap

Parquet support passes through `parquetjs-lite`. It provides file interoperability, not an Arrow-native or zero-copy data path. Parquet organizes storage into row groups, column chunks, and pages with defined encodings and compression [2]. Arrow defines a language-independent in-memory layout [1]. An optional columnar backend could share buffers among Parquet decoding, dataframe operations, and Wasm when alignment and ownership permit. Any zero-copy claim would first require buffer-identity and allocation measurements.

9 Threats to validity

Construct validity. The comparisons emphasize operations implemented by every participant and do not represent the entire dataframe task distribution. Canonical output hashing forces result materialization but does not serialize every result into an external format. The 505-name metric measures discoverability, not semantic compatibility. The browser study measures typed kernels, not the full DataFrame API.

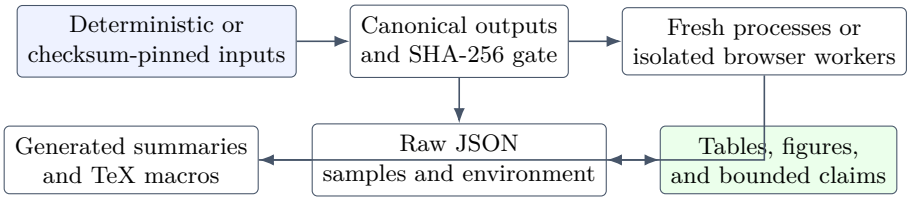


Fig. 8 Evidence path. A timing is reportable only after its output passes the reference digest and its raw record retains the execution context.

Package-size measurements externalize optional I/O dependencies and are not a full application bundle.

Internal validity. The main ablation and five-system study use fresh processes and randomized order, but they still run on an interactive workstation without fixed frequency, thermal, or power controls. Fixed warm-up counts are protocol choices rather than proof of steady state. Browser ratios have 30 independent contexts per engine and scale. The legacy Arquero and pandas timing tables remain single-process exploratory results.

External validity. Host-level results come from one Apple M5 Pro and Bun 1.4.0. They may not transfer to x86-64 Linux, different JavaScriptCore revisions, larger-than-memory data, or string-heavy workloads. The browser study broadens engine coverage but shares the same arm64 host. The compared systems have different semantics and representations; only the listed operations are matched.

Conclusion validity. The ablation intervals resample processes and iterations, and every timed backend passes a paired output digest. Browser intervals resample 30 independent contexts, while the five-system tables remain descriptive because their five-process replication is too small for strong interval claims. No result establishes a universal speedup. Native x86-64 replication and controlled Linux cgroup measurements would strengthen transferability and capacity analysis, but neither is part of the present result set.

10 Reproducibility and artifact

The repository includes the official December 2024 Springer Nature template archive with its SHA-256 digest, unmodified template sources, the manuscript, benchmark scripts, raw JSON samples, environment capture, package analysis, generated macros, a checksum-pinned public workload manifest, and Linux runners. Figure 8 shows how raw evidence reaches manuscript claims.

The minimal local reproduction sequence is:

```
bun install
bun run build:wasm
bun test
bun run conformance
bun run bench:fresh
bun run bench:competitors
bun run bench:browser
bun run workload:public
```

```
bun run bench:competitors:uci
bun run paper/artifact/analyze-package-size.ts
bun run paper/artifact/summarize-study.ts
bun run paper/artifact/summarize-expanded-study.ts
bun run artifact:checksums
```

The repository also includes an optional x86-64 route that builds a digest-pinned `linux/amd64` container and refuses to label a run as native unless `uname -m` reports `x86_64`. A separate cgroup runner can apply the same memory and memory-swap limit to each system, record current and peak memory, capture memory events, and report out-of-memory termination. Apple-hosted instruction emulation was rejected after Bun faulted under an emulated CPU lacking AVX; no emulated timing appears in the results. A separate cross-platform runner executes the correctness, package, conformance, host, public-data, and browser studies on macOS, Windows, or Linux and embeds their outputs in one returnable JSON report.

Raw artifacts retain every fresh-process iteration, correctness digest, process diagnostic, differential observation, mismatch category, and rejected-run reason. The repository includes commands for every reported measurement and follows the documentation, consistency, completeness, and exerciseability criteria used in artifact review [3]. A versioned archival DOI and independent clean-checkout reproduction remain pre-submission tasks. Cross-platform timing reports will remain separate from the macOS tables unless a later revision defines and executes a multi-host design.

11 Conclusion

`bun_panda` combines pandas-inspired TypeScript semantics with a small Rust/WebAssembly kernel set. The differential oracle reaches 100.00% agreement in the tested ten-family corpus under its declared value, label, dtype, exception, mutation, and ordering rules. The fresh-process ablation supports WebAssembly for full numeric sort and fused GroupBy over cached typed columns, but not for mask filtering or large top-1,000 selection. The five-system study finds favorable synthetic counting and filtering cells, no public operation-only win, and a persistent join deficit. The browser study then shows that even typed-kernel conclusions reverse across engines. Backend selection must account for semantics, operation shape, representation reuse, and runtime version. The current evidence package therefore makes macOS-scoped performance claims; cross-platform replication and controlled capacity measurement remain future validation work.

Acknowledgements. N/A.

Author contributions. Touhidul Alam Seyam is the sole author of the current manuscript.

Funding. N/A.

Data availability. Primary benchmark inputs are generated deterministically by scripts included in the artifact. The optional public-data validation uses the UCI Bank Marketing dataset under CC BY 4.0. Its DOI, source URL, field mapping, row count,

and SHA-256 checksums are fixed in the workload manifest. No private dataset is required.

Code availability. Source, raw measurements, and reproduction scripts are available in the immutable GitHub release v0.4.1, preserved by Software Heritage. The release identifier is swh:1:rel:a2e89f64f97deb08f9d8702ede3829acc719e92f. The corresponding software package is available from npm as bun_panda 0.4.1. The experiments report version 0.4.0 because that is the version recorded in the frozen raw measurements; version 0.4.1 packages the same evaluated implementation with the paper artifact and release metadata.

Conflict of interest. None.

References

[1] Apache Arrow Project (2025) Arrow columnar format. URL https://arrow.apache.org/docs/format/Columnar.html, format version 1.5

[2] Apache Parquet Project (2025) Apache parquet format specification. URL https://github.com/apache/parquet-format

[3] Association for Computing Machinery (2020) Artifact review and badging, version 1.1. URL https://www.acm.org/publications/policies/artifact-review-and-badging-current

[4] Barrett E, Bolz-Tereick CF, Killick R, et al (2017) Virtual machine warmup blows hot and cold. Proceedings of the ACM on Programming Languages 1(OOPSLA):1–27. https://doi.org/10.1145/3133876

[5] Data APIs Consortium (2023) Dataframe interchange protocol: Design requirements. URL https://data-apis.org/dataframe-protocol/latest/design_requirements.html

[6] Georges A, Buytaert D, Eeckhout L (2007) Statistically rigorous java performance evaluation. In: Proceedings of the 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications. ACM, pp 57–76, https://doi.org/10.1145/1297027.1297033

[7] Haas A, Rossberg A, Schuff DL, et al (2017) Bringing the web up to speed with webassembly. In: Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation. ACM, pp 185–200, https://doi.org/10.1145/3062341.3062363

[8] Heer J, Moritz D (2026) Arquero: Query processing and transformation of array-backed data tables. URL https://idl.uw.edu/arquero/, version 8.0.3

[9] Jangda A, Powers B, Berger ED, et al (2019) Not so fast: Analyzing the performance of webassembly vs. native code. In: 2019 USENIX Annual Technical Conference. USENIX Association, pp 107–120, URL https://www.usenix.org/conference/atc19/presentation/jangda

[10] JavaScript Data Science Community (2026) Danfo.js documentation. URL https://danfo.jsdata.org/

[11] Kalibera T, Jones RE (2013) Rigorous benchmarking in reasonable time. In: Proceedings of the 2013 International Symposium on Memory Management. ACM, pp 63–74, https://doi.org/10.1145/2464157.2464160

[12] Kohn A, Moritz D, Raasveldt M, et al (2022) Duckdb-wasm: Fast analytical processing for the web. Proceedings of the VLDB Endowment 15(12):3574–3577. https://doi.org/10.14778/3554821.3554847

[13] McKinney W (2010) Data structures for statistical computing in python. In: van der Walt S, Millman J (eds) Proceedings of the 9th Python in Science Conference, pp 56–61, https://doi.org/10.25080/Majora-92bf1922-00a

[14] Moro S, Rita P, Cortez P (2014) Bank marketing. https://doi.org/10.24432/C5K306, URL https://archive.ics.uci.edu/dataset/222/bank+marketing

[15] Mozzillo A, Zecchini L, Gagliardelli L, et al (2025) Evaluation of dataframe libraries for data preparation on a single machine pp 337–349. https://doi.org/10.48786/EDBT.2025.27

[16] npm, Inc. (2025) npm-pack documentation. URL https://docs.npmjs.com/cli/v11/commands/npm-pack/

[17] NVIDIA Corporation (2026) cudf pandas accelerator mode. URL https://docs.rapids.ai/api/cudf/stable/cudf_pandas/

[18] Oven, Inc. (2026) Bun runtime documentation. URL https://bun.com/docs/runtime, runtime version 1.4.0 used in the evaluation

[19] pandas development team (2026) pandas 3.0.5 api reference. URL https://pandas.pydata.org/docs/reference/

[20] Petersohn D, Macke S, Xin D, et al (2020) Towards scalable dataframe systems. Proceedings of the VLDB Endowment 13(11):2033–2046. https://doi.org/10.14778/3407790.3407807

[21] Petersohn D, Tang D, Durrani R, et al (2021) Flexible rule-based decomposition and metadata independence in modin: A parallel dataframe system. Proceedings of the VLDB Endowment 15(3):739–751. https://doi.org/10.14778/3494124.3494152

[22] Polars Contributors (2026) Polars: Node.js bindings. URL https://docs.pola.rs/user-guide/installation/

[23] Pyodide Project (2026) Pyodide documentation. URL https://pyodide.org/en/stable/

[24] World Wide Web Consortium (2025) Webassembly core specification 2.0. URL https://www.w3.org/TR/wasm-core-2/